

Interfaces in Java

What is an Interface?

- An interface is a blueprint of a class.
- It contains abstract methods and constants.
- It specifies what a class should do.
- A class must implement all abstract methods of an interface.
- All variables in an interface are public, static, and final by default.
- Interfaces can have default, static, and private methods (Java 8+ and 9+).

Defining an Interface

Interface Syntax in Java

An interface is declared using the interface keyword.

Syntax

X

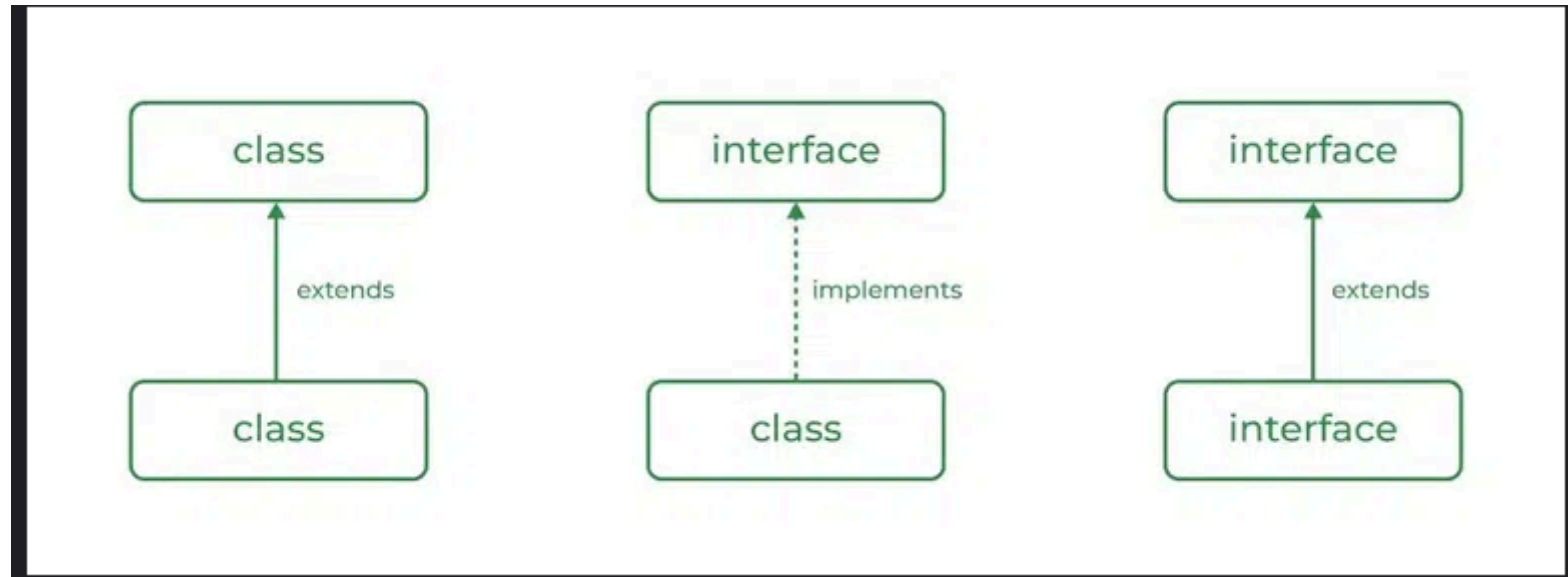
```
interface InterfaceName {  
    // constants  
    // abstract methods  
}
```

Example

```
interface Animal {  
    void sound(); // abstract method  
}
```

Relationship Between Class and Interface

- A class can extend another class and similarly, an interface can extend another interface. However, only a class can implement an interface and the reverse (an interface implementing a class) is not allowed.



Implementing an Interface

- A class uses 'implements' to provide method definitions.

Program: Implementing an Interface

```
interface Animal {  
    void sound();  
}
```

```
class Dog implements Animal {  
    public void sound() {  
        System.out.println("Dog barks");  
    }  
}
```

```
public class Main {  
    public static void main(String[] args) {  
        Dog d = new Dog();  
        d.sound();  
    }  
}
```

Output

Dog barks

Extending an Interface

- An interface can inherit another interface using 'extends'.

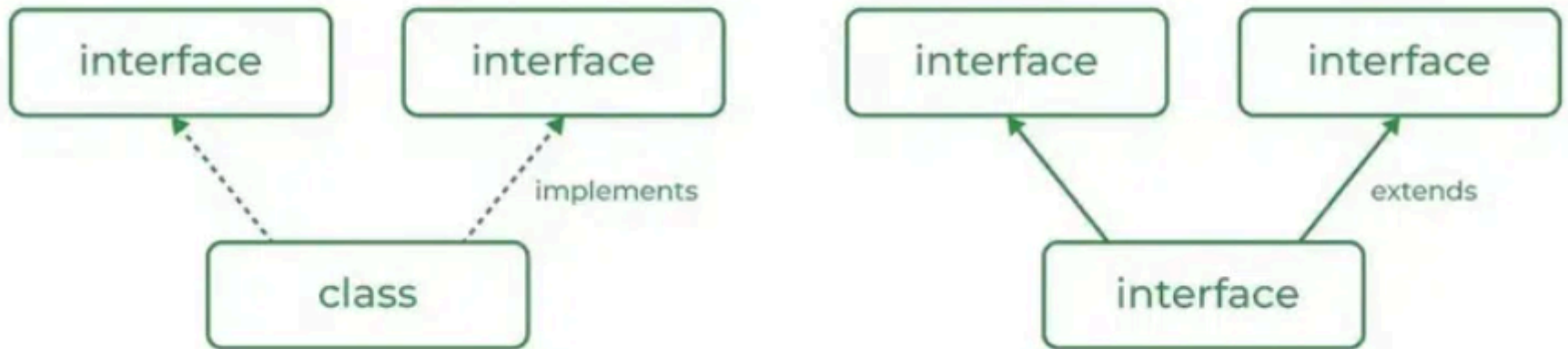
Program: Extending an Interface

```
interface Animal
{
    void sound();
}
interface Pet extends Animal
{
    void play();
}
class Dog implements Pet
{
    public void sound()
    {
        System.out.println("Bark");
    }
    public void play()
    {
        System.out.println("Play");
    }
}
```

Multiple Inheritance in Java Using Interface

Java does not support multiple inheritance with classes to avoid ambiguity, but it supports multiple inheritance using interfaces.

Multiple inheritance in Java



```
import java.io.*;

// Add interface
interface Add{
    int add(int a,int b);
}

// Sub interface
interface Sub{
    int sub(int a,int b);
}

// Calculator class implementing Add and Sub
class Cal implements Add , Sub
{
    // Method to add two numbers
    public int add(int a,int b){
        return a+b;
    }

    // Method to sub two numbers
    public int sub(int a,int b){
        return a-b;
    }
}
```

```
class GFG{  
    // Main Method  
    public static void main (String[] args){  
  
        // instance of Cal class  
        Cal x = new Cal();  
        System.out.println("Addition : " + x.add(2,1));  
        System.out.println("Substraction : " + x.sub(2,1));  
    }  
}
```

Output

```
Addition : 3  
Subtraction : 1
```

Interface Variables

- Variables declared in an interface are automatically: public, static, and final.

Program: Interface Variables

```
interface Circle {  
    double PI = 3.14159;  
}  
  
public class Main  
{  
    public static void main(String[] args)  
    {  
        System.out.println(Circle.PI);  
    }  
}
```

Output:
3.141459

Functional Interfaces

- A **Functional Interface** in Java is an interface that contains **exactly one abstract method**.
- It is also known as a Single Abstract Method (SAM) interface.
- Starting with Java 8, these interfaces provide the target types for lambda expressions and method references
- Use `@FunctionalInterface` annotation.

Declaring a Functional Interface

- To create a custom functional interface, use the standard interface keyword and ideally include the `@FunctionalInterface` annotation. This annotation is optional but highly recommended because it forces the compiler to flag an error if you accidentally add a second abstract method

Example:

```
@FunctionalInterface
interface Message
{
    void display();
}
```

@FunctionalInterface

public interface MyPrinter

{ // Exactly one abstract method

void printMessage(String message);

// Optional: Any number of default methods are allowed

default void logInfo(String info)

{

System.out.println("Log: " + info);

}

// Optional: Any number of static methods are allowed

static void printSystemTime()

{

System.out.println(System.currentTimeMillis());

}

}

Lambda Expressions

- A lambda expression is a concise way to implement a functional interface.

Syntax:

(parameters) -> expression

```

import java.util.ArrayList;
import java.util.List;
public class LambdaExample
{
// 1. Define a functional interface (Only ONE abstract method)
@FunctionalInterface
interface MathOperation
{
int operate(int a, int b);
}
public static void main(String[] args)
{
// 2. Multi-line lambda with 'return' keyword and curly braces
MathOperation addition = (a, b) ->
{
return a + b;
};
// 3. Single-line lambda (Implicit return, no curly braces required)
MathOperation multiplication = (a, b) -> a * b;
// 4. Executing the lambda expressions
System.out.println("10 + 5 = " + addition.operate(10, 5));    System.out.
println("10 * 5 = " + multiplication.operate(10, 5));    System.out.println("\n---
Built-in Collection Example ---");

```

```
// 5. Using a built-in lambda with Collection's forEach method
List<String> frameworkList = new ArrayList<>();    frameworkList.
add("Spring");
frameworkList.add("Hibernate");
frameworkList.add("Struts");
// Lambda takes a single parameter 'item' and prints it    frameworkList.
forEach(item -> System.out.println("Framework: " + item));
}
}
```

Outp

41
~~10~~ + 5 = 15

10 * 5 = 50

--- Built-in Collection Example ---

Framework: Spring

Framework: Hibernate

Framework: Struts

Summary

- Interface → Defines behavior
- implements → Class implementation
- extends → Interface inheritance
- Functional Interface → One abstract method
- Lambda → Short implementation syntax